Claude Code Build Spec — BWF Virtual Seats

This is the implementation handoff. Hand the whole `design/` folder to Claude Code along with `blue-for-bob-v4.html` (the visual reference / source for stand geometry). This spec is opinionated; deviate only with reason.

0. Read first

- `01-functional-design.md` — what we're building and why
- `02-brief-evaluation.md` — what's missing from the brief
- `blue-for-bob-v4.html` — the existing mock (canvas geometry + visual style baseline)
- `Asset Pack PCUK/` — BWF logo, Getty imagery, brand colours, stat tiles

1. Stack (locked)

Layer	Choice	Notes
Framework	Next.js 15, App Router, TS strict	Vercel-native
UI	Tailwind v4, Radix primitives where needed	No Material/Chakra
Forms	React Hook Form + Zod	Server-side validation with same Zod schemas
Database	Supabase Postgres	Row-level security on user-facing tables
Auth	Supabase Auth — magic link only	No passwords. Used for `/manage` and `/admin`
Realtime	Supabase Realtime	Broadcast channel for new claims
Storage	Supabase Storage	Avatar sprites, BWF assets, exported CSVs
Payments	Stripe — Embedded Checkout + Payment Intents	One-shot, no subscriptions
Email	Resend	Transactional only — receipts, magic links
Analytics	Vercel Analytics + Plausible (self-host or hosted)	GDPR-friendly, no cookies
Errors	Sentry	Free tier covers v1 traffic
Rate limit	Upstash Ratelimit (Redis)	On hold/donate/tribute endpoints
OG images	`@vercel/og`	Per-seat share cards
Avatars	DiceBear (lorelei-neutral or pixel-art), self-hosted	Swap for custom kit later if budget

2. Repo layout

```
bwf-virtual-seats/
├── app/
│   ├── (marketing)/
│   │   ├── page.tsx                  # Landing
│   │   ├── wall/page.tsx             # Tribute wall
│   │   ├── prize-terms/page.tsx      # Prize draw T&Cs (MDX)
│   │   ├── privacy/page.tsx
│   │   └── terms/page.tsx
│   ├── stadium/
│   │   ├── page.tsx                  # Canvas stadium + wizard
```

```
│   │   ├── components/
│   │   │   ├── StadiumCanvas.tsx
│   │   │   ├── SeatTooltip.tsx
│   │   │   ├── DonationWizard/
│   │   │   │   ├── index.tsx
│   │   │   │   ├── StepAmount.tsx
│   │   │   │   ├── StepDetails.tsx
│   │   │   │   ├── StepTribute.tsx
│   │   │   │   ├── StepAvatar.tsx
│   │   │   │   ├── StepGiftAid.tsx
│   │   │   │   └── StepPay.tsx
│   │   │   └── AvatarBuilder.tsx
│   │   └── geometry.ts               # STANDS, ROW_SP, vToRad, etc. — port from mock
│   ├── seat/[id]/
│   │   ├── page.tsx                  # Public seat card
│   │   └── og.png/route.ts           # Dynamic OG image
│   ├── manage/
│   │   ├── page.tsx                  # Donor dashboard (magic-link auth)
│   │   └── seats/[id]/edit/page.tsx
│   ├── admin/
│   │   ├── page.tsx                  # Dashboard
│   │   ├── tributes/page.tsx         # Moderation queue
│   │   ├── donations/page.tsx
│   │   ├── prize-draw/page.tsx
│   │   └── pricing/page.tsx
│   ├── api/
│   │   ├── holds/route.ts            # POST create hold, DELETE release
│   │   ├── checkout/route.ts         # POST create Stripe session
│   │   ├── stripe/webhook/route.ts   # POST Stripe webhook
│   │   ├── claims/[id]/route.ts      # GET, PATCH (donor edit)
│   │   ├── seats/route.ts            # GET seat manifest (cached)
│   │   ├── tributes/route.ts         # POST flag, admin moderate
│   │   ├── prize/draw/route.ts       # POST admin runs draw
│   │   └── admin/* (CSV export, pricing, etc.)
│   ├── layout.tsx
│   └── iframe-bridge.ts              # postMessage resize for iframe parent
├── lib/
│   ├── db/                           # Supabase client + typed queries
│   ├── stripe/                       # Stripe client + helpers
│   ├── email/                        # Resend templates
│   ├── avatars/                      # Sprite atlas + render helpers
│   ├── geometry.ts                   # Stand/seat coord helpers (shared)
│   ├── validators/                   # Zod schemas
│   ├── ratelimit.ts
│   └── moderation.ts                 # Profanity filter
├── supabase/
│   ├── migrations/                   # SQL migrations (see §4)
│   ├── seed.sql                      # Seed seats from STANDS geometry
│   └── functions/                    # Postgres functions for atomic claim-on-webhook
├── public/
│   └── assets/                       # BWF logo, sprites, etc.
├── tests/
│   ├── e2e/                          # Playwright — full donation flow
│   └── unit/                         # Vitest — geometry, moderation, validators
├── .env.example
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
```

```
└── package.json
```

3. Environment variables

```
# .env.example
# --- Public ---
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
NEXT_PUBLIC_SITE_URL=https://seats.bobwillisfund.org
NEXT_PUBLIC_PLAUSIBLE_DOMAIN=seats.bobwillisfund.org

# --- Server ---
SUPABASE_SERVICE_ROLE_KEY=        # Used only server-side, never exposed
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
RESEND_API_KEY=
RESEND_FROM_EMAIL=seats@bobwillisfund.org

UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

SENTRY_DSN=
SENTRY_AUTH_TOKEN=

# --- Admin allowlist ---
ADMIN_EMAILS=mark@example.com,adam@bobwillisfund.org,ricky@bobwillisfund.org

# --- Feature flags ---
ENABLE_AVATARS=true
ENABLE_PRIZE_DRAW=true
ENABLE_GIFT_AID=true
```

4. Database schema (SQL)

```
-- supabase/migrations/0001_init.sql

create extension if not exists "pgcrypto";
create extension if not exists "citext";

-- ============================================================================
-- SEATS — physical layout, seeded once from geometry
-- ============================================================================
create table seats (
  id              text primary key,                -- e.g. 'hollies_R_3_17'
  stand_id        text not null,                   -- 'hollies' | 'wyatt' | ...
  stand_name      text not null,
  row_index       int  not null,
  col_index       int  not null,
  tier            text not null check (tier in ('premium','standard','general')),
  base_price_pence int not null,                   -- £10 floor for everyone in v1; per-tier in admin
  x_coord         numeric(10,4),
  y_coord         numeric(10,4),
  is_disabled     boolean not null default false,
  created_at      timestamptz not null default now()
);
```

```
create index seats_stand_idx on seats(stand_id);

-- ============================================================
-- DONORS — one per email
-- ============================================================
create table donors (
  id                uuid primary key default gen_random_uuid(),
  email             citext unique not null,
  full_name         text,
  -- Gift Aid address
  address_line1     text,
  address_line2     text,
  city              text,
  postcode          text,
  is_uk_taxpayer    boolean,
  marketing_opt_in  boolean not null default false,
  created_at        timestamptz not null default now()
);

-- ============================================================
-- DONATIONS — Stripe payment record
-- ============================================================
create table donations (
  id                        uuid primary key default gen_random_uuid(),
  donor_id                  uuid not null references donors(id) on delete restrict,
  amount_pence              int  not null,                -- what they paid
  base_amount_pence         int  not null,                -- £10 (or seat tier)
  bump_amount_pence         int  not null default 0,
  gift_aid                  boolean not null default false,
  gift_aid_amount_pence     int  not null default 0,      -- 25% if gift_aid = true
  stripe_payment_intent_id  text unique,
  stripe_checkout_session_id text unique,
  status                    text not null check (status in
('pending','succeeded','failed','refunded')),
  refunded_at               timestamptz,
  created_at                timestamptz not null default now(),
  succeeded_at              timestamptz
);
create index donations_donor_idx on donations(donor_id);
create index donations_status_idx on donations(status);

-- ============================================================
-- CLAIMS — the canonical "this seat is taken"
-- ============================================================
create table claims (
  id                uuid primary key default gen_random_uuid(),
  seat_id           text unique not null references seats(id),  -- one claim per seat
  donor_id          uuid not null references donors(id),
  donation_id       uuid not null references donations(id),
  display_name      text,
  tribute_message   text check (length(tribute_message) <= 280),
  is_anonymous      boolean not null default false,
  is_amount_private boolean not null default false,
  avatar_config     jsonb,                                  -- see avatar schema in lib/avatars
  moderation_status text not null default 'auto_approved'
                    check (moderation_status in
('auto_approved','pending_review','approved','rejected')),
  moderation_notes  text,
```

```
  moderated_by        uuid,
  moderated_at        timestamptz,
  claimed_at          timestamptz not null default now(),
  updated_at          timestamptz not null default now()
);
create index claims_donor_idx on claims(donor_id);
create index claims_moderation_idx on claims(moderation_status);

-- ======================================================================
-- SEAT_HOLDS — soft lock during checkout
-- ======================================================================
create table seat_holds (
  id          uuid primary key default gen_random_uuid(),
  seat_id     text not null references seats(id),
  session_id  text not null,                                  -- anonymous browser session
  donor_email citext,                                          -- if known at hold time
  expires_at  timestamptz not null,
  released    boolean not null default false,
  created_at  timestamptz not null default now()
);
-- Only one active hold per seat
create unique index seat_holds_active_uniq
  on seat_holds(seat_id)
  where released = false and expires_at > now();

-- ======================================================================
-- PRIZE DRAW
-- ======================================================================
create table prize_entries (
  id           uuid primary key default gen_random_uuid(),
  donation_id  uuid references donations(id),    -- null if postal entry
  donor_id     uuid references donors(id),       -- null if anonymous postal
  source       text not null check (source in ('donation','postal')),
  eligible     boolean not null default true,
  created_at   timestamptz not null default now()
);
create index prize_entries_donation_idx on prize_entries(donation_id);
create index prize_entries_eligible_idx on prize_entries(eligible);

create table prize_draw_results (
  id                uuid primary key default gen_random_uuid(),
  drawn_at          timestamptz not null default now(),
  drawn_by          text not null,                          -- admin email
  total_entries     int not null,
  winning_entry_id  uuid not null references prize_entries(id),
  seed              text not null,                          -- record CSPRNG seed for audit
  notes             text
);

-- ======================================================================
-- AUDIT LOG
-- ======================================================================
create table audit_log (
  id          uuid primary key default gen_random_uuid(),
  actor       text,                                         -- email or 'system'
  action      text not null,
  target_type text,
  target_id   text,
```

```
  meta        jsonb,
  created_at  timestamptz not null default now()
);
create index audit_log_target_idx on audit_log(target_type, target_id);

-- ============================================================
-- STRIPE EVENTS — idempotency
-- ============================================================
create table stripe_events (
  id          text primary key,    -- evt_xxx
  type        text not null,
  payload     jsonb not null,
  processed_at timestamptz not null default now()
);

-- ============================================================
-- VIEWS
-- ============================================================

-- Public seat status — denormalised for the canvas
create or replace view v_seat_status as
select
  s.id,
  s.stand_id,
  s.row_index,
  s.col_index,
  s.tier,
  case
    when s.is_disabled then 'disabled'
    when c.id is not null then 'claimed'
    when h.id is not null then 'held'
    else 'empty'
  end as state,
  c.is_anonymous as claim_is_anonymous,
  c.display_name as claim_display_name,
  c.avatar_config as claim_avatar_config
from seats s
left join claims c on c.seat_id = s.id
  and c.moderation_status in ('auto_approved','approved')
left join seat_holds h on h.seat_id = s.id and h.released = false and h.expires_at > now();

-- Live counters
create or replace view v_campaign_stats as
select
  (select count(*) from claims) as seats_claimed,
  (select sum(amount_pence + gift_aid_amount_pence) from donations where status='succeeded') as
total_raised_pence,
  (select count(*) from prize_entries where eligible = true) as prize_entries,
  (select count(*) from seats where not is_disabled) as total_seats;

-- ============================================================
-- ROW-LEVEL SECURITY
-- ============================================================

-- Public read on the view; tables themselves locked down
alter table seats enable row level security;
alter table claims enable row level security;
alter table donations enable row level security;
```

```
alter table donors enable row level security;
alter table seat_holds enable row level security;

-- Public can read the seat status view (handled by granting select on view)
grant select on v_seat_status to anon, authenticated;
grant select on v_campaign_stats to anon, authenticated;

-- Donors can read their own data
create policy "donors_self_read" on donors for select
  using ( email = auth.jwt() ->> 'email' );
create policy "claims_self_read" on claims for select
  using ( donor_id in (select id from donors where email = auth.jwt() ->> 'email') );
create policy "claims_self_update" on claims for update
  using ( donor_id in (select id from donors where email = auth.jwt() ->> 'email') );

-- Service role bypasses RLS; used by server-only routes
```

```
-- supabase/migrations/0002_atomic_claim.sql
-- Called from the Stripe webhook to atomically convert a hold into a claim
create or replace function fn_complete_claim(
  p_seat_id        text,
  p_donor_email    citext,
  p_donor_name     text,
  p_address_line1  text,
  p_address_line2  text,
  p_city           text,
  p_postcode       text,
  p_is_uk_taxpayer boolean,
  p_marketing_opt_in boolean,
  p_amount_pence   int,
  p_base_amount_pence int,
  p_bump_amount_pence int,
  p_gift_aid       boolean,
  p_gift_aid_amount_pence int,
  p_stripe_pi_id   text,
  p_stripe_cs_id   text,
  p_display_name   text,
  p_tribute        text,
  p_is_anonymous   boolean,
  p_is_amount_private boolean,
  p_avatar_config  jsonb,
  p_moderation_status text
) returns jsonb
language plpgsql
security definer
as $$
declare
  v_donor_id    uuid;
  v_donation_id uuid;
  v_claim_id    uuid;
begin
  -- upsert donor
  insert into donors (email, full_name, address_line1, address_line2, city, postcode, is_uk_taxpayer,
marketing_opt_in)
  values (p_donor_email, p_donor_name, p_address_line1, p_address_line2, p_city, p_postcode,
p_is_uk_taxpayer, p_marketing_opt_in)
  on conflict (email) do update set
    full_name = coalesce(excluded.full_name, donors.full_name),
```

```
    address_line1 = coalesce(excluded.address_line1, donors.address_line1),
    address_line2 = coalesce(excluded.address_line2, donors.address_line2),
    city = coalesce(excluded.city, donors.city),
    postcode = coalesce(excluded.postcode, donors.postcode),
    is_uk_taxpayer = coalesce(excluded.is_uk_taxpayer, donors.is_uk_taxpayer)
  returning id into v_donor_id;

  -- record donation
  insert into donations (donor_id, amount_pence, base_amount_pence, bump_amount_pence,
                         gift_aid, gift_aid_amount_pence,
                         stripe_payment_intent_id, stripe_checkout_session_id,
                         status, succeeded_at)
  values (v_donor_id, p_amount_pence, p_base_amount_pence, p_bump_amount_pence,
          p_gift_aid, p_gift_aid_amount_pence,
          p_stripe_pi_id, p_stripe_cs_id,
          'succeeded', now())
  returning id into v_donation_id;

  -- create claim — fails if seat already claimed (unique on seat_id)
  insert into claims (seat_id, donor_id, donation_id, display_name, tribute_message,
                      is_anonymous, is_amount_private, avatar_config, moderation_status)
  values (p_seat_id, v_donor_id, v_donation_id, p_display_name, p_tribute,
          p_is_anonymous, p_is_amount_private, p_avatar_config, p_moderation_status)
  returning id into v_claim_id;

  -- release any holds on this seat
  update seat_holds set released = true where seat_id = p_seat_id and released = false;

  -- log prize entry
  insert into prize_entries (donation_id, donor_id, source) values (v_donation_id, v_donor_id,
'donation');

  return jsonb_build_object('claim_id', v_claim_id, 'donation_id', v_donation_id, 'donor_id',
v_donor_id);
end;
$$;
```

5. API contracts

All requests JSON, all responses JSON. Errors: `{ error: { code, message } }` with appropriate HTTP status.

`POST /api/holds`

Reserve a seat for 10 minutes.

```
Request:  { seatId: string }                  // requires X-Session-Id header
Response: { holdId: string, expiresAt: string }
Errors:   409 SEAT_TAKEN | 410 SEAT_DISABLED | 429 RATE_LIMIT
```

`POST /api/checkout`

Create a Stripe Checkout session and return its client secret.

```
Request: {
  holdId: string,
  amountPence: number,        // 1000 minimum, integer
```

```
  donor: {
    email: string,
    fullName?: string,
    address?: { line1, line2?, city, postcode },
    isUkTaxpayer?: boolean,
    marketingOptIn?: boolean,
  },
  giftAid: boolean,
  tribute: {
    displayName?: string,      // omitted/undefined → anonymous
    message?: string,
    isAnonymous: boolean,
    isAmountPrivate: boolean,
    avatarConfig?: AvatarConfig,
  }
}
Response: { clientSecret: string }
Errors:   400 VALIDATION | 410 HOLD_EXPIRED | 429 RATE_LIMIT
```

The wizard's full state is persisted in metadata on the Checkout session. Webhook reads it and calls `fn_complete_claim`.

`POST /api/stripe/webhook`

Stripe → us. Handle:

- `checkout.session.completed` — happy path → `fn_complete_claim`
- `payment_intent.payment_failed` — release hold, no claim
- `charge.refunded` — flag donation refunded, mark seat re-released (admin-only flow)

Idempotent on `event.id` via `stripe_events`.

`GET /api/seats`

Returns the seat manifest with current state. Cache: `revalidate=2`. Realtime channel sends deltas between revalidations.

```
Response: {
  stats: { seatsClaimed, totalRaisedPence, prizeEntries, totalSeats },
  seats: Array<{
    id, standId, row, col, tier, state: 'empty'|'claimed'|'held'|'disabled',
    claim?: { displayName?: string, avatarConfig?: AvatarConfig, isAnonymous: boolean }
  }>
}
```

`PATCH /api/claims/:id`

Donor edits their tribute. Magic-link auth. RLS enforces ownership.

```
Request:  { displayName?, message?, isAnonymous?, isAmountPrivate?, avatarConfig? }
Response: { ok: true }
```

Admin endpoints (all require admin role)

- `GET /api/admin/donations?cursor=...` — list, paginated
- `POST /api/admin/tributes/:id/moderate` — approve/reject
- `POST /api/admin/seats/:id/disable` — disable seat (e.g. refund)

- `POST /api/admin/prize/draw` — run prize draw (one-time per campaign)
- `GET /api/admin/exports/donors.csv` — full donor + Gift Aid CSV

6. Stripe configuration

- **Mode:** Payment Intents via Embedded Checkout (`mode: 'payment'`).
- **Currency:** GBP only.
- **Customer creation:** create-or-update Stripe Customer per donor email.
- **Metadata on session:** the entire wizard payload (seat, tribute, gift aid, etc.) — keep <50 keys / 500 chars per value. Store the heavy stuff (avatar JSON) in a Supabase `pending_claims` row keyed by `holdId` and only put the `holdId` in Stripe metadata. (Avoids Stripe metadata size limits.)
- **Receipt email:** Stripe's built-in receipt is OK as a backup; we send our own HTML receipt via Resend on `checkout.session.completed`.
- **Webhook endpoint:** `https://NEXT_PUBLIC_SITE_URL/api/stripe/webhook` — signing secret in `STRIPE_WEBHOOK_SECRET`.
- **Test mode first.** Build everything against test keys. Switch to live only when Adam approves.

7. Seeding the seats

```
// supabase/seed.ts
// Run once after migrations. Idempotent.
// Reads STANDS geometry (ported from blue-for-bob-v4.html) and
// inserts one row per (stand, row, col) into seats.
// Total expected ~3000-4000 seats — confirm with final geometry pass.
```

Geometry is in `app/stadium/geometry.ts`, imported by both the canvas renderer and the seed script. Single source of truth.

8. Build sequence

Stage 1 — foundations

- Repo scaffold, Next.js, Tailwind, Supabase, Stripe, Resend, Sentry wired
- Migrations 0001 + 0002 applied
- Seed script runs, ~3500 seats in DB
- Stadium canvas renders from `v_seat_status` (states only, no avatars yet)
- Donor wizard mocked (UI only, no payment)
- E2E test scaffolding (Playwright)

Stage 2 — payment + claim loop

- Holds API + concurrency tested (Playwright with 2 parallel sessions hitting same seat)
- Stripe Embedded Checkout integrated, webhook handler complete, idempotency proven
- `fn_complete_claim` exercised end-to-end on Stripe test cards
- Receipt email (Resend) sent on success
- Gift Aid step + HMRC declaration text + storage
- Realtime channel: claims appear on other browsers within 2s
- Magic-link auth for `/manage` working

- **Milestone:** a fictional donor can claim a seat end-to-end in test mode, see it on the wall, edit their tribute, share their seat.

Stage 3 — avatars, prize draw, share cards

- DiceBear avatar builder, config persisted, rendered on seat card
- Simplified avatar render in canvas (skin + top + cap colours)
- Prize draw engine (entries, draw API, T&Cs page)
- OG image per seat (`/seat/[id]/og.png`)
- Tribute moderation: profanity filter + admin queue
- Admin dashboard: counters, recent donations, moderation queue, CSV export
- Cookie consent + privacy + terms pages

Stage 4 — polish, perf, launch

- Mobile testing on real devices (Mark, Adam, Ricky)
- Load test (k6 or similar) — 500 concurrent users hitting `/stadium`
- Performance budget: LCP < 2.5s on 3G; canvas redraws < 16ms
- Iframe-safe: postMessage resize, sandboxed cookies
- Final copy pass with Adam
- Stripe live keys, DNS, DKIM/SPF
- Soft launch end of week 4 morning, monitor, full launch by end of day 30 May

9. Acceptance criteria (v1 done = ship)

- [ ] Donor on iPhone SE (small viewport) can complete the full flow in < 90 seconds
- [ ] Two simultaneous users hitting the same seat: exactly one charged, exactly one claim, the other gets a clean "try another" message
- [ ] Stripe webhook is idempotent: replaying the same event creates no duplicates
- [ ] Gift Aid declaration text is HMRC-current, address fields are stored, total raised includes gift aid uplift
- [ ] Tribute containing a slur is auto-quarantined and visible to admin within 5 seconds
- [ ] Magic-link to `/manage` lets a donor edit their tribute and re-share
- [ ] OG image renders at 1200×630 with avatar + seat + name + BWF brand
- [ ] Realtime: a new claim appears on another browser's stadium within 3 seconds
- [ ] Admin can: moderate, refund (disable seat), export donor CSV, run the prize draw with audit log
- [ ] Lighthouse mobile: Performance > 85, Accessibility > 95, Best Practices = 100
- [ ] No PCI scope on our infra (Stripe Checkout handles it)
- [ ] Privacy / Terms / Prize Terms / Cookie banner all live

10. Things Claude Code should NOT do

- Don't pick a different framework or DB. The stack is locked.
- Don't add an ORM (Prisma, Drizzle). Supabase's typed client + raw SQL for complex bits is enough and faster to reason about.
- Don't store rendered avatar PNGs. Always render from JSON config client-side.

- Don't store card numbers, CVCs, anything PAN-related. Anywhere. Ever.
- Don't build a custom auth system. Magic link only.
- Don't build a 3D stadium. The mock is 2D and that's deliberate.
- Don't add server-side telemetry that tracks individual donors without consent.
- Don't ship without RLS enabled on user tables.
- Don't ship without webhook idempotency proven via test.

11. Things to ask the human (Mark) when blocked

1. Stripe live-mode go-ahead.
2. Final BWF charity number for the footer.
3. Final Gift Aid HMRC declaration wording (Adam to confirm).
4. Postal address for prize-draw free-entry route.
5. Final domain decision (`seats.bobwillisfund.org` or iframe).
6. Tribute moderation policy (auto-publish vs. hold-for-review).
7. Whether sponsor seat blocks are needed for v1.

12. Definition of "iframe-safe"

The app must work both standalone and inside the WordPress iframe at `bobwillisfund.org/virtualseats`.

- All cookies set with `SameSite=None; Secure` so they work cross-origin.
- A `postMessage` height-broadcast on every layout change so the parent can resize the iframe.
- No top-level navigation that would break out of the iframe.
- All shareable URLs are absolute, pointing to the canonical app domain.
- A small `iframe-bridge.ts` script handles parent ↔ child messages (resize, deeplinks).

13. Testing strategy

- **Unit (Vitest):** geometry helpers, validators (Zod), moderation matcher, prize-draw RNG determinism (with seeded mode), Gift Aid amount calculation.
- **Integration:** route handlers with a mocked Supabase client + Stripe stripe-mock.
- **E2E (Playwright):** full donate flow on Chromium + Mobile Safari; concurrency test (two browsers, same seat); webhook replay; admin moderation; magic link.
- **Load (k6):** 500 concurrent users on `/stadium`, 50 concurrent in checkout.
- **Manual:** real-device run on iPhone SE, mid-tier Android, iPad.